

Fase 1: Valar Compilis

En la Fase 1 es començarà el disseny i la programació del procés principal del sistema, el **Maester**.

En aquesta etapa, l'objectiu és establir les bases del programa: la seva configuració inicial i la interfície d'usuari, sense entrar encara en la comunicació de xarxa. El procés Maester haurà de ser capaç de llegir els seus fitxers de configuració i respondre a un conjunt de comandes introduïdes per l'usuari (Veure Taula 1).

Fitxer de configuració – Maester (*maester.dat*)

Aquest fitxer de text contindrà la informació bàsica i la taula de rutes del Maester en el següent format:

- El nom del regne.
 - La ruta de la carpeta on es troben els arxius de l'usuari.
 - El número d'Envoys que tindrà.
 - La IP i port (cadascun en una línia que acaba amb un '\n') on el Maester escoltarà connexions.
 - Una secció que comença amb “--- ROUTES ---” i defineix les connexions.
- Algunes entrades tindran una IP i port coneguts, mentre que d'altres podran tenir *.*.* per a indicar que el regne existeix, però la seva ruta no és coneguda directament.

Exemple de fitxer de text de configuració a la Figura 3:

```
Dragonstone
/dragon
mont 3
192.168.1.3
9003
--- ROUTES ---
DEFAULT 192.168.1.5 9002
KingsLanding 192.168.1.5 9002
TheVale 192.168.1.3 9005
Driftmark *.*.*.* 0
```

Figura 3. Fitxer de configuració d'un procés Maester.

Fitxer d'inventari – Maester (*stock.db*)

Cada **Maester** gestionarà l'inventari del seu regne a través del **llibre de comptes oficial**. Aquest llibre no és un simple pergamí, està escrit en un codi

secret que només els Mestres poden desxifrar per a protegir els secrets comercials de la casa.

Tècnicament, és un **fitxer binari**. Cada entrada del llibre especifica un producte amb els següents camps:

- **Nom:** char name[100];
- **Quantitat:** int amount;
- **Pes:** float weight;

El registre d'inventari s'ha de conservar entre execucions. Qualsevol modificació d'estoc s'ha de reflectir al fitxer d'inventari, per tal de garantir-ne la persistència.

Terminal de Comandes

El procés Maester ha de gestionar un terminal interactiu. La major part del temps, mostrarà logs d'activitat de les peticions que rebi per ell, o que hagi d'enrutar al següent. En tot moment el terminal ha de mostrar un terminal, per que l'usuari pugui introduir unes comandes personalitzades. Aquestes comandes són *case insensitive*.

MANDA	DESCRIPCIÓ
T REALMS	Mostra tots els regnes llistats al fitxer de configuració.
DGE <REALM> <sigil.jpg>	Envia una petició d'aliança a un altre regne.
DGE RESPOND <REALM> ACCEPT / REJECT	Respon afirmativament o negativament a una petició d'aliança que has rebut.
T PRODUCTS <REALM>	Mostra la llista de productes a un regne (necessària al·liança prèvia).
T PRODUCTS	Mostra la llista de productes de no indicar regne, mostrarà els seus propis productes. No cal aliança prèvia, donat que són els productes del propi Maester.
RT TRADE <REALM>	Envia una sessió interactiva per a crear una "l·lista de la compra". Mostra els productes disponibles (obtinguts prèviament amb LIST PRODUCTS) i permet a l'usuari afegir-los a la l·lista.
DGE STATUS	Mostra l'estat de les teves aliances (enemics, aliats).
ROY STATUS	Mostra l'estat de tots els teus Envoys.
T	Finalitza l'execució.

Taula 1. Llistat de comandes esteses.

Per la comanda START TRADE, caldrà mostrar un menú com el que es mostra a l'exemple, per construir una "l·lista de la compra" a enviar al regne demanat.

Per aquesta primera fase, caldrà detectar les comandes d'aquest menú, amb productes inventats (podeu fer servir els vostres propis) i quan s'acabi una comanda, escriure la llista de compra en un fitxer. Aquest fitxer es farà servir més endavant per enviar-ho a altres regnes. El format d'aquest fitxer és lliure, però ha de ser un fitxer de text.

```
$montserrat:> Maester config.dat stock.db

Maester of Dragonstone initialized. The board is set.
$ PLEDGE TheVale sigil.jpg
Pledge sent to TheVale. Envoy 1 is on its way.
$ PLEDGE STATUS
- TheVale: PENDING
- KingsLanding: PENDING
- Driftmark: PENDING
$ ENVOY STATUS
- Envoy 1: ON_MISSION (PLEDGE to TheVale)
- Envoy 2: FREE
- Envoy 3: FREE
$ LIST PRODUCTS KingsLanding
ERROR: You must have an alliance with KingsLanding to trade.
$ PLEDGE KingsLanding sigil.jpg
Pledge sent to KingsLanding. Envoy 2 is on its way.
$ something else
Unknown command
$ ENVOY STATUS
- Envoy 1: ON_MISSION (PLEDGE to TheVale)
- Envoy 2: ON_MISSION (PLEDGE to KingsLanding)
- Envoy 3: FREE
$
>>> Alliance with The Vale forged successfully!
$ PLEDGE STATUS
- TheVale: ALLIED
- KingsLanding: ON_PENDING
- Driftmark: PENDING
$ LIST PRODUCTS TheVale
Listing products from TheVale:
  1. Myrish Lace (150 units)
  2. Sweetwine (80 units)
$ START TRADE TheVale
Entering trade mode with TheVale.
Available products: Myrish Lace, Sweetwine.
(trade)> add Myrish Lace 10
(trade)> add Sweetwine 5
(trade)> send
Trade list sent to TheVale.
. next figure
```

Figura 4.1. Execució d'un procés Maester

```

Trade list sent to TheVale.
$ PLEDGE Driftmark sigil.jpg
Pledge sent to Driftmark. An envoy is on its way.
$
>>> Pledge to King's Landing has failed (TIMEOUT).
$ PLEDGE STATUS
- TheVale: ALLIED
- KingsLanding: FAILED
- Driftmark: PENDING
$ LIST PRODUCTS
--- Trade Ledger ---
Item | Value (Gold) | Weight (Stone)
-----
Antlered Helm | 250 | 4.5
Warhammer of Bronze | 400 | 8.0
Stormlands Ale Barrel | 80 | 10.0
Ship Timber Plank | 60 | 15.0
Salted Fish Barrel | 55 | 20.0
Wine Cask | 120 | 5.0
Leather Riding Boots | 90 | 2.0
Bronze Helm | 140 | 3.5
Woolen Blanket | 30 | 2.5
Iron Dagger | 35 | 1.2
Grain Sack | 40 | 10.0
-----
Total Entries: 11

$ EXIT
The Maester of Dragonstone signs off. The ravens rest.

$montserrat:>

```

Figura 4.2. Execució d'un procés Maester.

Es demana:

- Dissenyar i programar l'executable maester.
- En iniciar-se, el Maester haurà de processar els seus fitxers de configuració (.dat i .db) i emmagatzemar la informació a les estructures de dades corresponents.
- El Maester haurà de reconèixer totes les ordres de la Taula 1. Per a aquesta fase:
 - S'implementarà la funcionalitat completa i local de les ordres: LIST REALMS, LIST PRODUCTS (sense el realm), EXIT i START TRADE.

- Per a la resta d'ordres, el programa simplement mostrarà "Command OK" per indicar que ha reconegut l'ordre.
- El sistema de reconeixement d'ordres haurà de ser robust. Si una ordre és incorrecta, mostrarà "Unknown command". Si és una ordre vàlida però incompleta (p. ex., PLEDGE sense arguments), mostrarà un missatge d'ajuda lliure com: *Did you mean to send a pledge? Please review syntax.*

Consideracions Fase 1:

- Un cop finalitzada l'execució de TOTS els processos s'ha d'**alliberar** tot tipus de memòria dinàmica significativa si n'hi hagués.
- En cas que els noms dels regnes continguin '&', s'hauran d'eliminar. Això és degut al protocol de comunicació del sistema, el qual està a l'Annex. Aquest serà *case sensitive*.
- Les comandes són *case insensitive*.
- Podem considerar que el **format del fitxer de configuració és correcte**.
- **No** es poden utilitzar les funcions *printf*, *scanf*, *gets*, *puts*, etc. Només es podrà interactuar amb la pantalla i amb fitxers amb les funcions *read* (lectura) i *write* (escriptura). Sí que es permet fer ús de la funció *asprintf* i similars.
- **No** es poden utilitzar les funcions *system*, *popen*, *stat* ni cap variant.
- Cal garantir l'estabilitat de l'aplicació i el seu correcte funcionament. En cap cas es poden produir bucles infinits, *core dumped*, esperes actives, *warnings* al compilar, etc. També cal controlar tots aquells aspectes susceptibles de donar algun error i, en cas que es produeixin, **informar degudament a l'usuari** i, si és possible, seguir amb el funcionament normal de l'aplicació.
- Des d'ara i fins al final de la pràctica cal considerar que el procés Maester pot finalitzar la seva execució utilitzant la comanda adequada o prement CTRL+C. Cal tenir-ho en compte i procurar perquè tot el sistema quedi el més **estable** possible en cas que es produeixi. Alliberant els recursos i deixant tot correctament tancat.
- És obligatori la utilització d'un *makefile* per generar l'executable.
- No és suficient que l'arxiu que pugeu al pou tingui extensió .tar, sinó que s'ha de poder desempaquetar amb la comanda tar. Qualsevol pràctica o *checkpoint* que no es pugui "descomprimir" d'aquesta manera **no serà corregida**.
- Recordeu que el codi ha d'estar degudament modul·lat, és a dir: no es pot ubicar tot en un sol fitxer .c, i és obligatori que hi hagi un fitxer *makefile*. Tingueu en compte que si en intentar corregir una pràctica aquesta no compila amb la comanda *make* (pel motiu que sigui), l'entrega serà qualificada com a no apta (2).
- Juntament amb el codi, cal entregar una explicació de com s'ha dissenyat i estructurat la fase. **És obligatori que, abans de començar a programar**, feu aquest anàlisi i **ho valideu amb els monitors de l'assignatura. Reviseu l'Annex III per a més detalls**. L'informe ha d'incloure de manera clara i entenedora:
 - Diagrames que expliquin els processos que s'han creat.
 - Estructures de dades usades i la seva justificació.

Fase 2: El Camí Reial

En aquesta segona fase caldrà implementar les connexions entre els diferents regnes del sistema. Els processos Maester es començaran a enviar les primeres comunicacions per a forjar aliances i, posteriorment, comerciar.

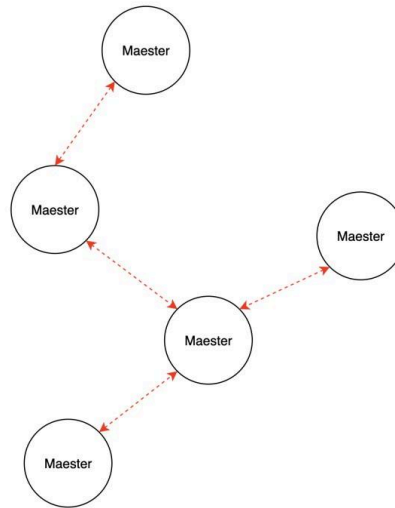


Figura 5. Esquema funcional a dissenyar e implementar

Cada Maester actuarà com a un node d'una xarxa distribuïda, encarregant-se tant de gestionar les seves pròpies comunicacions com d'actuar de node intermedi (hop) per a les comunicacions dels d'altres regnes veïns. Com que els diversos processos Maester poden estar en diferents màquines físiques, aquesta connectivitat caldrà implementarla mitjançant **sockets**, i seguint el **protocol de comunicació** que trobareu a l'Annex.

Per defecte, els regnes no tenen cap aliança preestablerta, però l'usuari en podrà crear des de cada procés Maester. Per fer-ho, els Maesters envien un segell (una imatge) que certifica que és un missatge oficial. El procediment a seguir és:

1. Un Maester "A" inicia una petició d'aliança amb un altre regne "B" mitjançant un primer missatge (TYPE 0x01). Si "A" no té a "B" a la seva taula de rutes, haurà d'enviar el missatge per tots els hops que calgui.
2. Posteriorment, el Maester "B" confirma que està llest per rebre el segell (TYPE 0x31).
3. A continuació, el Maester "A" fa tot l'enviament del segell (TYPE 0x02).
4. El Maester "B" decideix si accepta o denega l'aliança (TYPE 0x03).
5. Finalment, el Maester "A" ha d'enviar una trama de confirmació ACK (TYPE 0x31) notificant que ha rebut la resposta. Si la resposta arriba transcorreguts més de dos minuts s'enviarà una trama NACK (TYPE 0x69) indicant que l'aliança queda descartada a causa del TIMEOUT.

No obstant, **en aquesta fase 2** simplificarem el procés per forjar aliances, i no implementarem l'enviament de les dades de la imatge del segell, només el *header* (missatge TYPE 0x01). És a dir, quan es vulgui crear una aliança entre dos regnes, el primer Maester enviarà la trama de petició d'aliança cap al segon, aquest respondrà amb la confirmació/denegació de l'aliança, i per últim, el primer regne enviarà ACK/NACK per indicar que ha rebut la resposta o si l'ha descartat per TIMEOUT, però en cap moment s'enviarà la imatge del segell (missatges TYPE 0x31 i 0x02). Això s'implementarà a la **fase 3**.

Connectivitat del sistema i enrutament de missatges (hops)

Per que tota la xarxa de comunicació funcioni correctament, quan un Maester rep un missatge, ha de determinar si és el destinatari final o si ha d'actuar com a node intermedi (hop) i reenviar-lo a un altre regne. Per fer-ho seguirà el següent procediment:

1. Quan el camp DESTINATION és el meu regne:
 - a. Revisar que el missatge estigui correcte (usant el valor de *checksum*).
 - i. Si és erroni, enviar un NACK (TYPE 0x69) a qui m'hagi enviat el missatge i descartar-lo.
 - b. Si és correcte, enviar un ACK corresponent (en funció del tipus de missatge) a qui m'hagi enviat el missatge.
 - c. Processar el missatge.
2. Si el camp DESTINATION no és el meu regne:
 - a. Revisar que el missatge estigui correcte (usant el valor de *checksum*).
 - i. Si és erroni, enviar un NACK (TYPE 0x69) a qui m'hagi enviat el missatge (hop anterior) i descartar-lo.
 - b. Buscar a la taula de rutes a qui va dirigit.
 - c. Si existeix la ruta, reenviar-lo al IP:port indicat.
 - d. Si no existeix, reenviar-lo a la ruta DEFAULT.
 - e. Si no hi ha ruta DEFAULT, mostrar l'error i descartar el missatge (el node origen detectarà l'error per *timeout*).
3. El node origen esperarà un total de 2 minuts a rebre resposta del regne de destí.
Si aquesta no es produeix, assumirà que hi ha hagut algun error i donarà timeout.

Cal tenir en compte que, a l'inici, totes les comunicacions es faran amb aquest mecanisme. És a dir, si un regne "A" ha d'enviar un missatge a "C" passant pel regne "B", totes les comunicacions per establir aliança hauran de passar per aquest regne "B" intermedi (menys l'ACK final, que va directament). No obstant,

quan ja s'hagi establert l'aliança entre "A" i "C", aquests regnes podran comunicar-se directament (sense passar per "B"), utilitzant els corbs i optimitzant el comerç.

Comerçant amb els aliats

Un cop s'ha establert una aliança amb un altre regne, es podrà comerciar amb aquest (altrament, es denegaran totes les trames relacionades amb el comerç). El comerç es basa en demanar les llistes de productes i fer intercanvis de bens. Tot i això, en **aquesta fase 2** simplificarem aquesta operativa, i totes les trames de comerç retornaran un ACK vàlid, però sense processar internament el missatge ni actualitzar l'estoc. Això ho reservem per la **fase 3**.

Finalment, quan un procés Maester decideix acabar la seva execució, ha d'avisar a tots els regnes amb qui està aliat (TYPE 0x27).

<pre>\$montserrat:> Maester dragonstone.dat stock.db Maester of Dragonstone initialized. The board is set.\$ PLEDGE TheVale sigil.jpg Pledge sent to TheVale. Envoy 1 is on its way. >>> Alliance with TheVale forged successfully! \$ LIST PRODUCTS TheVale Direct route available (allied). No hops required. List products: OK (Pending F3) \$ EXIT The Maester of Dragonstone signs off. The ravens rest. \$montserrat:></pre>	<pre>\$montserrat:> Maester kingslanding.dat stock.db Maester of KingsLanding initialized. The board is set. >>> Received hop: Dragonstone → TheVale Found route: TheVale → 192.168.1.4:9005 Forwarding PLEDGE request...</pre>
<pre>\$matagalls:> Maester vale.dat stock.db Maester of TheVale initialized. The board is set. >>> Alliance request received from Dragonstone.\$ PLEDGE RESPOND Dragonstone ACCEPT Alliance with Dragonstone established. >>> Trade request received from Dragonstone (direct).Order processed successfully. Stock updated.</pre>	

Figura 6. Exemple d'execució amb 3 Maesters ("A" i "B" a montserrat; "C" a matagalls), i on "A" forja l'aliança amb "C", passant per "B".

Consideracions Fase 2:

- Les de la Fase 1 segueixen essent vàlides.
- En aquesta Fase NO s'ha d'implementar la transferència d'imatges (segells) ni les comandes de comerç.
- Caldrà gestionar les caigudes de tots els processos, i mantenir l'estabilitat del sistema (enviant missatges d'error quan les coses no funcionin)
- Juntament amb el codi, cal entregar una explicació de com s'ha dissenyat i estructurat la fase. És molt recomanable que, abans de començar a programar, feu aquest anàlisi i ho valideu amb els monitors de l'assignatura. **Reviseu l'Annex IV per a més detalls.** L'informe ha d'incloure de manera clara i entenedora:
 - Diagrames que expliquin els processos que s'han creat, les diferents comunicacions entre processos, etc.
 - Estructures de dades usades i la seva justificació.
 - Recursos del sistema utilitzats (*threads, forks, signals, sockets, semàfors, pipes, etc.*) amb la seva justificació.
 - Opcionals implementats.

Fase 3: La dansa dels segells

En aquesta fase es desplegarà una de les funcionalitats més esperades del **Citadel System**: la transferència de fitxers entre regnes. Després d'haver establert la connectivitat bàsica i les primeres aliances a la fase anterior, ara serà el moment de posar en circulació els **segells** i els **fitxers comercials** (llistes de productes i comandes) que sostenen les relacions diplomàtiques i econòmiques entre les Grans Cases.

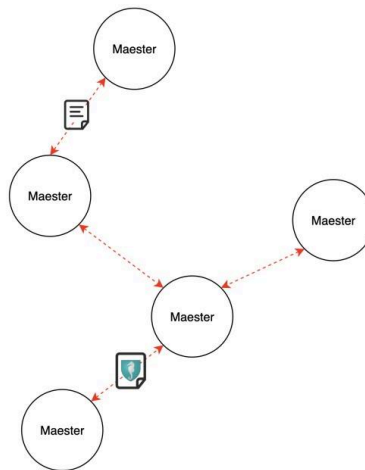


Figura 7. Esquema funcional del enviament de segells i comerç

Cada Maester haurà de ser capaç no només d'enviar i rebre aquests fitxers, sinó també de verificar-ne la integritat. Per a això es farà servir la comanda md5sum del sistema operatiu, que permet comprovar que el contingut rebut coincideix exactament amb l'original. Només si la verificació és correcta es considerarà vàlida la petició d'aliança o la transacció comercial.

El procés de transferència s'articula en dues parts diferenciades:

- Inici de transferència (header): s'envia la informació essencial del fitxer (nom, mida i valor del MD5SUM).
- Enviament de dades: s'envia el fitxer fragmentat en trames de dades.

A la fase anterior, s'haurà fet l'enviament del header, ara tocarà ampliar-ho enviant també les dades. Un cop completada la transferència, el regne receptor ha de respondre amb un missatge de validació, confirmant si el fitxer rebut és vàlid o si s'ha corromput.

La funcionalitat de Fase 3 cobreix dos escenaris principals:

- Aliances: enviament del fitxer sigil.jpg que simbolitza l'acord entre dues cases.
- Comerç: enviament de llistes de productes i comandes com a fitxers.

Les decisions de disseny d'aquesta comunicació caldrà explicar-les de manera adient a la memòria de la pràctica, tot fent un diagrama de la comunicació implementada.

A més, aquesta fase també introdueix la gestió de l'inventari de cada regne. Cada Maester haurà de ser capaç de consultar el seu llibre de comptes (stock.db) i respondre adequadament a les peticions d'aliats. Si un regne rep una comanda i no disposa de prou estoc, haurà de respondre amb un rebuig. Si el producte no existeix, també ho haurà d'indicar amb la trama adequada.

Com que les comunicacions del Citadel System es fan mitjançant hops, tots aquests fitxers circularan a través de regnes intermedis, que no modificaran ni el camp ORIGIN ni el camp DESTINATION, sinó que simplement retransmetran la trama. Si no hi ha ruta coneguda, el node intermedi haurà de respondre amb una trama d'error. En cas que ja siguin aliats, podran comunicar-se de manera directa, donat que en aliar-se s'han compartit les seves IP + Port.

Durant aquesta fase, per tant, veurem per primer cop com els Maesters esdevenen veritables nodes de comunicació fiable: capaços de moure informació binària entre ells, de validar-la i de reflectir els resultats en el seu inventari local.

<pre>tserrat:> Maester Dragonstone.dat stock.db ter of Dragonstone ialized. The board is EDGE TheVale sigil.jpg ge sent to TheVale. Envoy on its way. Alliance with TheVale ed successfully! PART TRADE TheVale ct route available ied). No hops required. ring trade mode with Vale. roducts available. Use PRODUCTS first. de)> exit ST PRODUCTS TheVale ontinue on next page</pre>	<pre>gpedros:> Maester KingsLanding.dat stock.db ter of KingsLanding initialized. The board is ived hop: Dragonstone → TheVale (PLEDGE) d route: TheVale → 168.1.4:9005Forwarding...</pre>
--	---

ing products from ale: . Myrish Lace (150 units) . Sweetwine (80 units) ART TRADE TheVale ect route available ied). No hops required. ring trade mode with ale. lable products: Myrish , Sweetwine. de)> add Myrish Lace 10 de)> add Sweetwine 5 de)> send e list sent to TheVale. Order accepted by ale. IT Maester of Dragonstone s off. The ravens rest. gonstone>	
agalls:> Maester vale.dat stock.db ter of TheVale initialized. The board is set. Alliance request received from onstone. \$ PLEDGE RESPOND Dragonstone PT Alliance with Dragonstone established. LIST PRODUCTS request from Dragonstone. ing product ..Products delivered. Trade request received from Dragonstone. r processed successfully. Stock updated.	

Figura 8. Exemple d'execució: 3 Maesters ("A" a montserrat, "B" a puigpedros; "C" a matagalls). "A" comercia amb "C", i passa per "B" només per la aliança.

Consideracions Fase 3:

- Les de les fases anteriors segueixen essent vàlides.
- Els fitxers d'aliança (segells) i comerç (l·listes i comandes) s'han de transferir íntegrament i verificar amb **md5sum**.
- No es permès usar codi programat per fer l'MD5SUM. Cal executar la comanda **md5sum** que proporciona el mateix *bash* del sistema operatiu (*md5sum*).
- Els regnes han d'actualitzar l'inventari local segons les comandes acceptades, reflectint la modificació persistent a *stock.db*.
- No es pot utilitzar memòria secundària addicional per a emmagatzemar estat temporal de les transferències més enllà dels fitxers propis del sistema.
- Juntament amb el codi, cal entregar una explicació de com s'ha dissenyat i estructurat la fase. **Reviseu l'Annex III per a més detalls**. L'informe ha d'incloure de manera clara i entenedora:
 - Diagrames que expliquin processos s'han creat, les diferents comunicacions entre processos, etc.
 - Estructures de dades usades i la seva justificació.
 - Recursos del sistema utilitzats (*threads*, *forks*, *signals*, *sockets*, semàfors, *pipes*, etc.) amb la seva justificació.
 - Opcionals implementats.

Fase 4: El consell dels Emissaris

En la fase final de la pràctica, s'amplia la capacitat d'acció del regne. Fins ara, un Maester ja gestionava múltiples tasques en paral·lel: la seva funció principal era rebre constantment els corbs que arribaven, ja fos per a reexpedir-los cap al següent salt de la ruta com un "router", o per a processar-los si eren per al seu propi regne. A més d'aquesta tasca constant de recepció i enrutament, el Maester només podia iniciar i gestionar una única missió pròpia a la vegada, ja fos una petició d'Aliança o de Comerç.

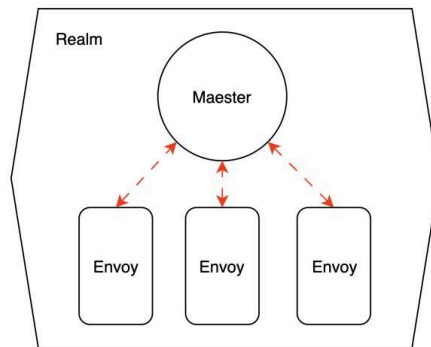


Figura 9. Esquema funcional dels processos d'un regne

En aquesta fase, implementarem el Consell d'Emissaris (Envoys) per a eliminar aquesta limitació. L'objectiu és permetre que un Maester pugui enviar múltiples corbs propis simultàniament, multiplicant la seva capacitat per a forjar aliances i comerciar.

Cada Envoy serà un procés independent, instanciat pel Maester principal. El nombre d'Envoys és un paràmetre del fitxer de configuració. Quan el jugador vulgui iniciar una missió, el Maester buscarà un Envoy lliure i li assignarà la tasca. Aquest Envoy s'encarregarà de tota la gestió de la comunicació en quant a la tasca assignada, alliberant el Maester per a continuar amb les seves funcions de gestió i enrutament.

Consideracions Fase 4:

- Les de les Fases anteriors segueixen essent vàlides.
- No es poden emprar sockets ni memòria secundària per la comunicació Envoy <-> Maester.
- Cal garantir l'exclusió mútua amb tots els recursos que puguin ser compartits.

Requeriments de lliurament i planificació

Aquesta pràctica disposarà de diferents **punts de control o checkpoints** per poder ferne un seguiment acurat i garantir la consolidació de cadascuna de les fases de manera incremental. Concretament se seguirà el següent calendari de dates límit:

CONCEPTE	DE AD LIN E
Fase 1	26/ 10/ 202 5

Fase 2	23/ 11/ 202 5
Fase 3	14/ 12/ 202 5
Lliurament Final 1 (sobre 10)	06/ 01/ 202 6
Lliurament Final 2 (sobre 10)	01/ 02/ 202 6
Lliurament Final 3 (sobre 8)	17/ 05/ 202 6
Lliurament Final 4 (sobre 6)	14/ 06/ 202 6

Els *checkpoints* no són obligatoris, tot i ser altament recomanables per poder garantir la robustesa de la pràctica i per saber que aneu en els terminis correctes per poder lliurar al final del semestre. Aquests *checkpoints* ponderen un 10% de la qualificació de la pràctica.

També és **obligatori validar el disseny global de la pràctica** amb els monitors de pràctiques abans d'iniciar la fase 1. Així podreu garantir que la implementació no patirà d'inconsistències insalvables per fases posteriors. Per això cal aprofitar els horaris de dubtes i **portar els dissenys** a validar. Per més informació consulteu Annex III. Cal recordar que una pràctica funcioni no és cap garantia que sigui vàlida, doncs pot ser que tot el disseny no compleixi els requisits de l'enunciat. Per això és molt important tenir garanties sobre el disseny a implementar.

Els lliuraments es realitzaran a l'eStudy en un pou amb un fitxer que inclogui el codi font (fitxers .c, .h i el makefile) de la pràctica, funcionant completament sobre Montserrat, així com els fitxers de dades i configuració utilitzats. El fitxer haurà de ser obligatòriament en format .tar. El podeu generar amb la comanda següent:

```
tar cf Gx_Fn.tar *.c *.h makefile *.ext_fitxers
```

on Fn és el número de Fase a lliurar. Per exemple, el grup 12 enviaria G12_F1.tar per al lliurament del *checkpoint* de la Fase 1.

Per l'entrega final del Gener en endavant, n és 4, per la Fase 4.

Qualsevol entrega que no segueixi el format d'entrega no serà corregida.

En els lliuraments parcials també cal incloure la memòria que es va realitzant per a que es pugui valorar la seva evolució. A partir de la Fase 2 cal que aquesta contingui

obligatòriament el disseny de la pràctica.

En els lliuraments finals també caldrà dipositar **memòria final en format PDF** en el tar. La memòria ha de constar, obligatòriament, dels punts que s'indiquen a continuació en l'Annex I.

Annex I: Contingut de la memòria

La memòria ha de ser una documentació externa correctament maquetada. Ha de contenir els següents apartats:

1. **Portada**
2. **Índex**
3. **Disseny:** explicació de com s'ha dissenyat i estructurat la pràctica. Es pot explicar per fases o de la pràctica en global. Ha d'incloure de manera clara i entenedora:
 - a. Diagrames que expliquin processos s'han creat, les diferents comunicacions entre processos, etc.
 - b. Estructures de dades usades i la seva justificació.
 - c. Recursos del sistema utilitzats (*signals*, *sockets*, semàfors, *pipes*, etc.) amb la seva justificació.
 - d. Opcionals implementats.
4. **Problemes observats i com s'han solucionat.**
5. **Estimació temporal:** temps dedicat en el desenvolupament total de la pràctica per a cadascun dels estudiants. Les categories a tenir en compte són:
 - a. Investigació: temps dedicat a buscar eines i/o a aprendre components a implementar (fora del temps dedicat a les sessions).
 - b. Disseny: temps dedicat a dissenyar la pràctica.
 - c. Implementació: temps de codificació.
 - d. *Testing*: temps dedicat a fer proves.
 - e. Documentació: temps dedicat a escriure la memòria i a la documentació interna del codi (comentaris, etc.).
6. **Conclusions i propostes de millora.**

7. (Opcional) Explicació de TOTS els noms dels processos de la pràctica. En quin tema ens hem basat aquest any per a fer l'enunciat?

8. **Bibliografia utilitzada** (en un format bibliogràfic correcte IEEE, APA, etc.): tant recursos bibliogràfics com enllaços.

Es valorarà la redacció i la correctesa gramatical i ortogràfica. La memòria ha de tenir **numeració de pàgines**.

La memòria s'ha d'entregar en **format PDF** juntament amb el lliurament final però es recomana anar-la elaborant durant el transcurs de la pràctica.

Annex II: protocol de comunicació general

Per dur a terme la comunicació entre processos en diferents màquines, s'utilitzarà un protocol específic d'enviament de trames que s'explicarà a continuació. S'ha de tenir en compte que els missatges s'enviaran mitjançant *sockets* utilitzant una comunicació orientada a connexió.

En aquest protocol s'utilitzarà un **únic tipus** de trama. Sent aquestes de dimensió **fixa (320 Bytes)** i sempre formades pels següents 6 camps:

Type: Descriu el tipus de la trama en format hexadecimal, 1 caràcter que contindrà un identificador de trama.

Origin: Camp de 20 bytes amb IP + Port del Regne que origina la trama.

Destination: Camp de 20 bytes amb el nom del Regne destí.

Data Length: Camp de 2 bytes en format numèric que serveix per indicar la llargària del camp *data*.

Data: Aquest camp serveix per emmagatzemar-hi valors o dades que ha d'enviar la trama. La llargària d'aquest camp depèn directament de la llargada total de la trama i el valor de *data_length*.

Checksum: Camp de 2 bytes que ajudarà a validar que la trama no ha tingut problemes. Aquest camp es calcularà sumant tots els bytes de la trama (sense comptar el propi checksum) i fent mòdul 2^{16} . Serà important omplir cada trama enviada amb aquest checksum, i validar-lo en arribar als respectius servidors.

TYPE (1 Byte)	ORIGIN (20 Bytes)	DESTINATION (20 Bytes)	DATA_LENTH (2 Bytes)	DATA (320 - 45 - X Bytes)	CHECKSUM (2 Bytes)
------------------	----------------------	---------------------------	-------------------------	---------------------------------	-----------------------

És molt important de cara a garantir la bona comunicació amb el procés que la definició de l'estructura de la trama de comunicació sigui estrictament, en ordre i format, la descrita amb anterioritat. Això vol dir que qualsevol canvi de tipus dels camps o ordre dels mateixos dins de l'estructura a dissenyar farà que les trames rebudes o enviades a procés no compleixin el protocol de comunicació i, per tant, no funcioni aquesta comunicació correctament. Si la mida de les dades a enviar és inferior a 320, caldrà omplir la trama amb el *padding* corresponent. És del tot normatiu l'enviament de trames serialitzades senceres en una única escriptura al socket. És a dir, llegir amb un sol read, i escriure amb un sol write de 320B. Per **totes les trames**, el camp CHECKSUM es calcularà de la següent manera:

CHECKSUM: Suma de tots els bytes de la trama % (2¹⁶). Podeu fer servir el valor 65536 directament. (Suma bytes % 65536). però per compatibilitat utilitzar el mòdul 32768.

Per simplicitat, no s'han inclòs a les explicacions individuals de cada trama donat que es faria repetitiu, però han de ser-hi a totes. Tot el que està entre <> ha de substituir-se pel valor corresponent. **Els <> NO van a la trama.**

PETICIÓ D'ALIANÇA (HEADER) - MAESTER

Maester A -> Maester B

Quan un Maester vol iniciar una aliança, primer envia la informació general i la del fitxer del segell

- TYPE: 0x01
- ORIGIN: <IP:Port del regne origen de la petició (no del hop)>
- DESTINATION: <Nom del regne destí> ▪ DATA_LENGTH: Llargària de les dades.
- DATA: <RealmName>&<SigilName>&<FileSize>&<MD5SUM>

On:

- RealmName: el nom del regne que origina la petició.
- SigilName: nom del fitxer del segell.
- FileSize: en bytes expressat en format ASCII.

- MD5SUM: 32 caràcters del MD5SUM.

En acabar d'enviar una trama de tipus 0x01, cal enviar la trama **ACK FITXER** (0x31).

NOTA: Aquesta serà la trama amb que es farà la F2. A partir de la F3 caldrà enviar també les dades del fitxer a continuació amb la trama:

ENVIAMENT DE SEGELL(DADES) - MAESTER

Maester A -> Maester B

- TYPE: 0x02
- ORIGIN: <IP:Port del regne origen de la petició (no del hop)>
- DESTINATION: <Nom del regne destí>
- DATA_LENGTH: Llargària de les dades. ▪ DATA: *dades_del_fitxer*.

Després d'enviar una trama de tipus 0x02, sempre caldrà enviar una trama de tipus **ACK MD5SUM** (0x32).

RESPOSTA A PETICIÓ D'ALIANÇA - MAESTER

Maester B -> Maester A

Quan un Maester rep una petició d'aliança i el md5sum està OK, respondrà amb una acceptació o un rebuig. Aquesta tindrà el format:

- TYPE: 0x03
- ORIGIN: <IP:Port del regne origen, de qui contesta a la petició d'aliança>
- DESTINATION: <Nom del Regne que demana la aliança> ▪ DATA_LENGTH: Llargària de les dades.
- DATA: <ACCEPT/REJECT>&<RealmName> On:
- RealmName: el nom del realm enviant la resposta (el mateix realm al qual corresponen IP:Port a l'ORIGIN).

És important que en aquest pas, la IP d'origen sigui la IP de Maester B, qui accepta. En aquesta resposta, el regne que ha iniciat la petició d'aliança ha de poder conèixer la IP + Port del aliat.

PETICIÓ DE LLISTA DE PRODUCTES - MAESTER

Maester A -> Maester B

Quan un Maester vol consultar els productes disponibles a un regne aliat, envia una petició inicial.

- TYPE: 0x11
- ORIGIN: <IP:Port del regne origen de la petició (no del hop)>
- DESTINATION: <Nom del regne destí> ▪ DATA_LENGTH: Llargària de les dades.
- DATA: <RealmName>

On:

- RealmName: el nom del regne que origina la petició.

RESPOSTA LLISTA DE PRODUCTES (HEADER) – MAESTER

Maester B → Maester A

El regne aliat prepara un fitxer amb la seva llista d'inventari i envia la trama d'inici.

- TYPE: 0x12
- ORIGIN: <IP:Port del regne aliat, que respon a la petició> ▪ DESTINATION: <Nom del regne origen, que ha iniciat la petició> ▪ DATA_LENGTH: Llargària de les dades.
- DATA: <FileName>&<FileSize>&<MD5SUM> On:
 - FileName: pot ser “products.db” o equivalent.
 - FileSize: mida en bytes, ASCII.
 - MD5SUM: 32 caràcters ASCII.

En acabar d'enviar una trama de tipus 0x12, cal enviar la trama **ACK FITXER** (0x31).

ENVIAMENT LLISTA DE PRODUCTES (DADES) – MAESTER

Maester B → Maester A

Després de la trama d'inici, el Maester aliat envia el fitxer amb la llista d'inventari en blocs binaris.

- TYPE: 0x13
- ORIGIN: <IP:Port del regne aliat> ▪ DESTINATION: <Nom del regne origen> ▪ DATA_LENGTH: Llargària de les dades. ▪ DATA: *dades_del_fitxer*.

Després d'enviar les trames de tipus 0x13, caldrà rebre la trama **ACK MD5SUM** (0x32).

PETICIÓ DE COMANDA (SHOPPING LIST HEADER) – MAESTER

Maester A → Maester B

Quan un Maester vol enviar una comanda, primer envia la informació del fitxer amb la llista de la compra.

- TYPE: 0x14
- ORIGIN: <IP:Port del regne origen> ▪ DESTINATION: <Nom del regne aliat>
- DATA_LENGTH: Llargària de les dades.
- DATA: <FileName>&<FileSize>&<MD5SUM>

En acabar d'enviar una trama de tipus 0x14, caldrà esperar la trama **ACK FITXER** (0x31).

ENVIAMENT DE COMANDA (DADES) – MAESTER

Maester A → Maester B

Després de la trama d'inici, el Maester origen envia el fitxer amb la llista de productes desitjats.

- TYPE: 0x15
- ORIGIN: <IP:Port del regne origen>
- DESTINATION: <Nom del regne aliat>

- DATA_LENGTH: Llargària de les dades.
- DATA: dades_del_fitxer

Després d'enviar les trames de tipus 0x15, caldrà rebre la trama **ACK MD5SUM** (0x32).

RESPOSTA A COMANDA – MAESTER

Maester B → Maester A

Quan el regne aliat processa la comanda rebuda, respon amb l'estat:

- TYPE: 0x16
- ORIGIN: <IP:Port del regne aliat> ▪ DESTINATION: <Nom del regne origen> ▪
- DATA_LENGTH: Llargària de les dades.
- DATA: OK o REJECT&<reason>

Exemple de reason:

- OUT_OF_STOCK si no hi ha prou quantitat d'algun producte.
- UNKNOWN_PRODUCT si no existeix al catàleg.

DESCONNEXIÓ DE MAESTER – MAESTER A → MAESTER B (aliat)

Quan un Maester finalitza la seva execució, ha d'avisar a tots els regnes amb qui té una aliança activa que es desconnecta.

- TYPE: 0x27
- ORIGIN: <IP:Port del regne que es tanca>
- DESTINATION: <Nom del regne aliat>
- DATA_LENGTH: Llargària de les dades.
- DATA: DISCONNECT

Comportament esperat:

- Els regnes aliats marquen l'aliança com a inactiva.
- No cal ACK de resposta: és una notificació.

ERROR: REGNE DESCONEGUT – NODE INTERMEDI → ORIGIN

Quan no existeix cap ruta específica ni DEFAULT cap al DESTINATION.

- TYPE: 0x21

- ORIGIN: <IP:Port del node intermedi que detecta l'error>
- DESTINATION: <Nom del regne origen de la petició> ▪ DATA_LENGTH: Llargària de les dades.
- DATA: UNKNOWN_REALM&<RealmName>

On:

- RealmName: el nom del regne que es desconeix.

ERROR: NO AUTORITZAT / SENSE ALIANÇA – RECEPTOR → ORIGIN

Quan es demana llista o comanda sense aliança prèvia o permisos.

- TYPE: 0x25
- ORIGIN: <IP:Port del receptor> ▪ DESTINATION: <Nom del regne origen> ▪ DATA_LENGTH: Llargària de les dades.
- DATA: AUTH&<RealmName>

On:

- RealmName: el nom del regne sense aliança.

PING (opcional per diagnòstic) – ORIGIN ↔ DESTINATION

Per comprovar liveness si ho necessites en F2/F3.

- TYPE: 0x26
- ORIGIN: <IP:Port de qui envia>
- DESTINATION: <Nom del regne destí>
- DATA_LENGTH: Llargària de les dades.
- DATA: PING o PONG

ACK FITXER - MAESTER

Maester B -> Maester A

Trama **OK** connexió. Indicant que pot començar a enviar l'arxiu.

- TYPE: 0x31
- ORIGIN: Buit
- DESTINATION: Buit
- DATA_LENGTH: Llargària de les dades
- DATA: OK<RealmName>

Trama **KO** connexió. S'envia en el cas que no s'hagi pogut establir la connexió.

El Maester desistirà d'enviar el arxiu.

- TYPE: 0x31
- ORIGIN: Buit
- DESTINATION: Buit
- DATA_LENGTH: Llargària de les dades
- DATA: KO<RealmName>

On:

- RealmName: el nom del regne que envia l'ACK.

ACK MD5SUM - MAESTER

Maester B -> Maester A

En acabar l'enviament d'un fitxer, caldrà que Maester B comprovi que el md5sum del fitxer rebut és el mateix que el d'origen. Caldrà contestar al emissor del fitxer amb la trama següent:

Comprovació del MD5SUM **correcta**.

- TYPE: 0x32
- ORIGIN: Buit
- DESTINATION: Buit
- DATA_LENGTH: Llargària de les dades.
- DATA: *CHECK_OK*<RealmName> Comprovació de MD5SUM **incorrecta**.

- TYPE: 0x32
- ORIGIN: Buit ▪ DESTINATION: Buit
- DATA_LENGTH: Llargària de les dades.
- DATA: *CHECK_KO*<RealmName>

On:

- RealmName: el nom del regne que envia l'ACK.

Procediment de detecció de trames errònies

Si qualsevol dels processos del sistema Citadel rep una trama que no es correspon a cap dels formats definits o amb un Checksum que no és vàlid caldria que s'enviés una trama NACK

de retorn per poder detectar l'error amb la següent trama:

- TYPE: 0x69
- ORIGIN: Built
- DESTINATION: Built
- DATA_LENGTH: Llargària de les dades.
- DATA: <RealmName>

Tots els errors de xarxa han de mostrar: 'Els corbs s'han perdut - Error [codi]'

Annex III: Guia per a un bon disseny de una pràctica

Quan treballes en un sistema complex com el Citadel System, és essencial aplicar un bon disseny per gestionar la complexitat de manera eficient i escalable. El disseny ha de comptar amb conceptes clau com la modularització, la concurrència i la robustesa. Com a enginyers, és fonamental tenir clar el disseny abans de començar la implementació, ja que un plantejament ben estructurat ens permetrà detectar possibles problemes, optimitzar recursos i garantir que el projecte sigui mantenible a llarg termini. A continuació, es presenten punts generals a tenir en compte en qualsevol projecte en C, centrats en els aspectes importants d'un sistema operatiu i com es poden aplicar al context del projecte.

Modularització: Disseny de mòduls compartits

En projectes complexos com el "**Citadel System**", modularitzar correctament el codi és crucial per garantir la claredat, la reutilització i la mantenibilitat. No es tracta només de dividir el sistema en processos independents com **Maester** i **Envoys**, sinó també d'identificar funcionalitats comunes i encapsular-les en **mòduls compartits**. Aquests mòduls, utilitzats per diferents processos, són essencials per evitar la duplicació de codi i assegurar una major eficiència.

En qualsevol projecte de certa complexitat, és fonamental determinar quines parts del codi poden ser comunes entre els diferents processos i encapsular-les de manera independent. Si diverses funcionalitats es repeteixen en diferents punts del sistema, com la manipulació de dades o l'accés a recursos compartits, és molt més eficient organitzar-les en mòduls reutilitzables. Això facilita la mantenibilitat del sistema, ja que redueix la duplicació de codi i centralitza els canvis: qualsevol modificació en un mòdul es propagarà automàticament als processos que l'utilitzin, estalviant temps i errors potencials.

Cada **mòdul** hauria de tenir una **responsabilitat clara** i ser fàcilment reutilitzable sense modificacions per part dels processos que l'utilitzen. Això no només millora l'estructura del codi, sinó que permet **centralitzar actualitzacions** i mantenir un sistema més net, escalable i fàcil de gestionar.

Un bon disseny modular en C implica la separació del codi en **fitxers .h** i **fitxers .c**. Els **fitxers .h** contenen declaracions de funcions, constants i tipus de dades que poden ser compartits entre mòduls o processos, mentre que els **fitxers .c** implementen la lògica funcional. Aquesta separació no només facilita l'organització del projecte, sinó que també permet que el compilador gestioni millor les dependències, cosa que millora la integració i compilació del codi.

A l'hora de modularitzar, és important seguir aquests **criteris**:

1. **Reutilització:** Identifica les funcionalitats necessàries en diferents processos i encapsula-les en mòduls reutilitzables. Això evita la duplicació de codi i fa que el sistema sigui més fàcil de mantenir.
2. **Clarificació de responsabilitats:** Cada mòdul ha de tenir una responsabilitat única i clara. No barregis funcionalitats diferents dins d'un mateix mòdul per evitar confusions i complicar la seva reutilització.
3. **Independència dels mòduls:** Els mòduls no han de dependre fortament entre ells. Si hi ha dependències, han d'estar ben definides i documentades per evitar problemes futurs d'integració o manteniment.

Finalment, és important evitar l'ús de **variables globals** dins dels mòduls compartits. Les variables globals haurien de residir únicament en els processos principals com **Maester.c**, mentre que els mòduls han de funcionar únicament amb funcions que rebin tots els arguments necessaris com a paràmetres. Això no només millora la modularitat del codi, sinó que també ajuda a evitar conflictes quan aquests mòduls són utilitzats per diversos processos en diferents contextos del sistema.

Concurrencia: Apropar-se al món real

En el disseny d'un sistema operatiu o d'un sistema complex com el "**Citadel System**", és fonamental entendre que, en el món real, moltes tasques han de succeir simultàniament per garantir el funcionament correcte del sistema. La **concurrència** permet que múltiples usuaris, processos o recursos treballin alhora, cosa que fa que el sistema pugui respondre a múltiples peticions de forma eficient.

Per exemple, si diversos usuaris estan fent peticions al mateix temps (com els processos **Envoy**), la concurrència assegura que aquestes es puguin atendre simultàniament, evitant colls d'ampolla i millorant l'eficiència general del sistema. A més, és possible que un mateix procés hagi de rebre, processar i enviar respostes al mateix temps. Això exigeix un disseny que permeti gestionar diverses operacions paral·leles sense interferir entre elles.

A l'hora de dissenyar un sistema concurrent, pots fer-te preguntes per garantir que estàs prenent decisions ben fonamentades com:

1. Com gestionaràs múltiples connexions simultànies?

Quina és la millor estratègia per gestionar múltiples usuaris alhora? Com asseguraràs que el sistema no es bloqueja si augmenta el nombre de peticions?

2. Com garantiràs la robustesa del sistema?

Què passarà si un fil o procés falla mentre està atenent una petició? Com asseguraràs que el sistema pugui continuar funcionant sense interrupcions? Quines tècniques de recuperació automàtica pots implementar per mantenir la continuïtat del servei?

3. Com controlaràs l'accés a recursos compartits

Si diversos fils o processos han d'accedir al mateix recurs alhora, com gestionaràs aquest accés de manera segura? On creus que podrien sorgir conflictes d'accés simultani?

4. Com optimitzaràs l'ús dels recursos del sistema?

Estàs fent un ús òptim de la CPU i la memòria? Com pots assegurar-te que no estàs generant més processos o fils dels necessaris? Quin impacte pot tenir sobre el sistema la creació massiva de fils o processos? Com pots evitar la saturació de recursos?

5. Quin és el millor enfocament per cada part del sistema?

Cada part del sistema necessita el mateix tipus de solució concurrent o és possible que diferents parts del sistema requereixin enfocaments diferents? Què seria més òptim per les diferents tasques que s'han de realitzar?

En el disseny concurrent d'un sistema no es tracta només de garantir que funcioni correctament, sinó que funcioni **bé sota càrrega** i en condicions **reals**. La clau està en trobar solucions que equilibren **robustesa** i **eficiència**, permetent que el sistema sigui flexible i resisteixi diferents situacions. Això implica reflexionar contínuament sobre les preguntes adequades durant el disseny de cada part del sistema, per trobar l'enfocament més adequat en funció dels requisits de **rendiment** i **estabilitat**.

Com a enginyer, has de ser capaç d'analitzar **críticament** cada component del sistema i adaptar-te als requisits particulars de cada procés. Això significa buscar el millor compromís entre **rendiment**, **robustesa** i **eficiència**, tenint en compte que el que és òptim per una part del sistema pot no ser-ho per una altra. La **concurrència** és una eina poderosa, però la seva correcta aplicació depèn de comprendre a fons les necessitats del sistema. La solució perfecta en un context pot no ser aplicable a altres escenaris, i part de la teva tasca és ajustar i adaptar aquestes solucions per assegurar que cada part funcioni de manera òptima en el seu context particular.